

Vertex Plugin API Specification

Version: 1.0

Status: Draft

1. Objetivo

A Plugin API é a interface pública oficial entre o Vertex Core e os plugins.

Seu objetivo é fornecer um contrato estável, seguro e versionado para que plugins possam utilizar os serviços da plataforma sem acessar implementações internas do Core.

Toda interação entre plugins e o Core deve ocorrer exclusivamente através da Plugin API.

2. Responsabilidades

A Plugin API é responsável por:

- expor serviços do Core aos plugins;
- ocultar detalhes de implementação;
- garantir compatibilidade entre versões;
- validar chamadas antes da execução;
- padronizar o acesso aos subsistemas;
- facilitar evolução da plataforma sem quebrar plugins existentes.

A Plugin API não implementa lógica de negócio; ela atua como camada de acesso.

3. Arquitetura

Plugin

↓

Plugin Context

↓

Plugin API

↓

Core API

↓

Core Services

A Plugin API funciona como uma camada de abstração entre plugins e o Core.

4. Componentes

Plugin Context

Ponto de entrada para todas as APIs disponíveis ao plugin.

Service Facades

Interfaces públicas para os serviços do Core.

Exemplos:

- Scheduler API;
- Storage API;
- Logging API;
- State API;
- Event API;
- Capability API;
- Resource API;
- UI API.

API Validator

Valida chamadas antes de encaminhá-las ao Core.

Verifica:

- permissões;
- argumentos;
- versão da API;
- estado do plugin.

Compatibility Layer

Mantém compatibilidade entre diferentes versões da Plugin API.

Error Mapper

Converte erros internos do Core em exceções padronizadas para os plugins.

5. Fluxos

Chamada de API

Plugin



Plugin Context



Plugin API



Core API



Serviço



Resultado

Erro

Plugin



Plugin API



Validação



Erro Padronizado

Verificação de Permissão

Plugin



Plugin API



Security Framework



Autorizado



Execução

6. APIs

A Plugin API é organizada por módulos.

Lifecycle API

- currentState()
- requestRestart()
- requestStop()

Scheduler API

- execute()
- schedule()
- cancel()

Storage API

- read()
- write()
- delete()
- exists()

Configuration API

- get()
- set()
- observe()

Logging API

- debug()
- info()
- warning()
- error()

State API

- publishState()
- consumeState()
- observeState()

Event API

- publishEvent()
- subscribe()
- unsubscribe()

Capability API

- registerCapability()

- unregisterCapability()
- discoverCapabilities()
- invokeCapability()

Resource API

- resourceUsage()
- remainingBudget()

UI API

- registerScreen()
- showScreen()
- closeScreen()

Asset API

- loadAsset()
- releaseAsset()

Android Bridge API

- requestAndroidService()
- openSystemSettings()
- requestPermission()

7. Estados

A Plugin API acompanha o estado do plugin que a utiliza.

Unavailable

↓

Available

↓

Restricted

↓

Suspended

↓

Closed

Chamadas realizadas em estados inválidos devem resultar em erro padronizado.

8. Políticas

API Pública

Plugins devem utilizar apenas APIs públicas documentadas.

Compatibilidade

Mudanças incompatíveis exigem nova versão principal da Plugin API.

Segurança

Toda chamada passa pela validação de permissões.

Estabilidade

Métodos obsoletos devem permanecer disponíveis por um período de transição sempre que possível.

Desempenho

A Plugin API deve introduzir o mínimo de sobrecarga possível entre o plugin e o Core.

9. Integrações

A Plugin API integra-se com:

- Core API;
- Plugin Manager;
- Scheduler;
- Resource Manager;
- Memory Management;
- Security Framework;
- Policy Engine;
- Storage Engine;
- Configuration System;
- Logging Framework;
- Capability System;
- Event System;
- State Engine;
- UI Runtime Engine;
- Asset Manager;
- Android Bridge.

Ela não acessa diretamente implementações privadas dos subsistemas; toda comunicação ocorre por contratos internos da Core API.

10. Métricas

A Plugin API registra:

- quantidade de chamadas por módulo;
- tempo médio de resposta;
- chamadas rejeitadas;
- erros por tipo;
- uso por plugin;
- versões da API em utilização;
- chamadas depreciadas;
- falhas de permissão.

Essas métricas auxiliam na evolução da plataforma e na identificação de gargalos.

11. Exemplos

Leitura de Configuração

1. Plugin solicita uma configuração.
2. Plugin API valida a chamada.

3. Configuration System retorna o valor.
4. Resultado é entregue ao plugin.

Publicação de Evento

1. Plugin publica um evento.
2. Plugin API valida o formato.
3. Event System distribui aos inscritos.

Invocação de Capability

1. Plugin descobre uma capability.
2. Plugin invoca a capability.
3. Plugin API verifica permissões.
4. Capability System executa a operação.
5. Resultado retorna ao plugin.

Escrita em Armazenamento

1. Plugin solicita gravação.
2. Plugin API verifica permissões e orçamento.
3. Storage Engine realiza a operação.
4. Resultado é retornado.

12. Regras Arquiteturais

- Plugins nunca acessam diretamente o Core.
- Toda chamada passa pelo PluginContext.
- A Plugin API deve permanecer estável entre versões compatíveis.
- Métodos devem ser independentes de implementações internas.
- Erros devem ser padronizados.
- APIs devem ser assíncronas sempre que envolverem operações potencialmente demoradas (como armazenamento, rede ou Android Bridge).

13. Limitações

A Plugin API não é responsável por:

- executar plugins;
- gerenciar ciclo de vida;
- controlar recursos;
- aplicar políticas de segurança;
- implementar serviços do Core.

Essas responsabilidades pertencem aos respectivos subsistemas.

14. Futuras Extensões

A Plugin API poderá evoluir para incluir:

- descoberta dinâmica de novos módulos de API;
- negociação automática de versão entre plugin e Core;
- geração automática de documentação e SDK;
- ferramentas de inspeção em tempo de execução;
- APIs experimentais isoladas das APIs estáveis.

Essas evoluções devem preservar a compatibilidade e a simplicidade de uso.

15. Resumo

A Plugin API é o contrato público do Vertex entre plugins e o Core. Ela fornece acesso consistente, seguro e versionado aos serviços da plataforma, isolando os plugins das implementações internas e permitindo que o Core evolua sem comprometer a compatibilidade do ecossistema. A Plugin API é um dos pilares da arquitetura modular do Vertex.